

HELIX

Cybersecurity Operations Platform — Proof of Concept

Architecture · Design · UML Documentation

Software Design Document

Cahier des Charges Technique

Author	Yasseene
Institution	Tek-Up University, Ariana, Tunisia
Academic Year	2025 – 2026
Version	1.0 – Proof of Concept
Classification	Academic — Confidential
Repository	github.com/mUchiha26/HELIX-PoC
Date	June 2026

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Engineering in Computer Science (Cybersecurity Track)

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Context and Problem Statement	3
1.3	Objectives	3
1.4	Scope	3
2	Technology Stack	4
3	Requirements Analysis	5
3.1	Functional Requirements Summary	5
3.2	Non-Functional Requirements Summary	6
3.3	User Roles and Permissions	6
4	Use Case Modelling	7
4.1	Global Use Case Diagram	7
4.2	Major Use Case Descriptions	9
5	Static Design	10
5.1	Class Diagram	10
5.2	Main Classes and Responsibilities	12
6	Dynamic Design	12
6.1	Authentication Workflow	13
6.2	Security Dashboard and Event Monitoring	15
6.3	Log Ingestion and ML Detection Pipeline	17
6.4	AI Assistant Workflow	19
6.5	Red Team Reconnaissance Workflow	21
7	Business Process Modelling	22
8	Software Architecture	26
8.1	Architectural Overview	26
8.2	Major Components	26
9	Deployment Architecture	27
9.1	Execution Environment and Node Responsibilities	27
10	Conclusion	28

10.1 Summary of the PoC 28

10.2 Achieved Objectives 28

10.3 Architectural Strengths 29

1. Introduction

1.1 Project Overview

HELIX is a modular cybersecurity operations platform that unifies four operational domains within a single authenticated environment: Blue Team defensive monitoring, Red Team offensive testing, AI-assisted threat analysis, and machine-learning anomaly detection. This Software Design Document (SDD) defines the architectural and design specifications for the Proof of Concept (PoC) phase, covering static structure, dynamic behaviour, deployment topology, and all major UML artefacts.

1.2 Context and Problem Statement

Global cybercrime costs continue to rise whilst security education environments remain fragmented across disjoint tools — separate SIEMs, scanners, AI assistants, and training platforms each requiring distinct credentials, data models, and skill sets. Students and early-career professionals consequently lack holistic, realistic environments for developing combined defensive and offensive competencies.

How to design and implement a unified, accessible cybersecurity platform that combines threat monitoring, AI-assisted analysis, and ML anomaly detection for academic and training purposes, while maintaining a modular architecture for future enterprise evolution?

1.3 Objectives

- **General:** Deliver a functional PoC demonstrating end-to-end platform viability within a unified web interface deployable on a standard XAMPP stack.
- **Functional:** Implement RBAC authentication, centralised dashboards, log ingestion, alert visualisation, ML anomaly scoring (Isolation Forest), AI-powered alert explanation (OpenAI API), and a Red Team network port scanner.
- **Academic:** Demonstrate the full software engineering lifecycle, multi-language integration (PHP, Python, Java, JavaScript), and produce a comprehensive UML documentation suite.

1.4 Scope

The PoC covers seven core modules: Authentication & RBAC, Security Dashboard, Log Ingestion, Alert Visualisation, ML Anomaly Detection, AI Assistant, and the Red Team Port

Scanner. Advanced exploitation frameworks, microservice orchestration, real-time streaming, and multi-tenancy are deferred to the MVP and production phases.

2. Technology Stack

The stack was selected to maximise accessibility and academic reproducibility whilst supporting all required capabilities.

Layer	Technology	Role and Justification
Frontend	HTML5, CSS3, JavaScript	Role-appropriate dashboards, data charts, and alert views. No framework required; served directly by Apache.
Backend / API	PHP 8.x + Composer	RESTful API, session management, RBAC enforcement, and orchestration of ML / AI sub-services.
Database	MySQL 8.x (XAMPP)	Relational persistence for users, logs, alerts, anomaly scores, chat history, and scan results.
ML Service	Python 3.x, scikit-learn	Isolation Forest unsupervised anomaly detection; invoked by PHP via <code>shell_exec()</code> .
Java Module	Java JDK	Hash scanning and log-parsing utilities; invoked by PHP <code>JavaBridge</code> service.
AI Integration	OpenAI API	Natural-language alert explanation and remediation guidance over HTTPS.
Server / Env.	Apache 2.x (XAMPP)	Local development and PoC deployment environment.

Table 1: HELIX PoC Technology Stack

The PHP backend follows a strict layered architecture: Front Controller → Middleware (`AuthMiddleware`, `RoleMiddleware`) → Controllers → Services → Models → PDO/MySQL. Composer provides PSR-4 autoloading. No layer may be skipped; controllers never access the database directly.

3. Requirements Analysis

3.1 Functional Requirements Summary

IDs	Module	Key Requirements
FR-001–007	Authentication & RBAC	Registration, bcrypt login ($\geq$ cost 10), session management, role assignment, account enable/disable, admin user management.
FR-010–014	Dashboard	Aggregated statistics, severity charts, time-series views, role-filtered metrics.
FR-020–024	Log Ingestion	CSV/TXT upload, field parsing, bulk DB insertion, duplicate detection, ingestion summary.
FR-030–034	Alert Visualisation	Severity-coded listing (Critical/High/Medium/Low), filter by source/time, acknowledgement workflow.
FR-040–044	ML Anomaly Detection	Isolation Forest scoring on event frequency, error rate, IP diversity; threshold-based alert generation; score history.
FR-050–054	AI Assistant	Multi-turn conversational analysis, alert context attachment, remediation suggestions, session persistence.
FR-060–063	Red Team Tools	Port scan with target/range specification, result persistence, scan history; restricted to Red/Purple Team roles.

Table 2: Functional Requirements by Module

3.2 Non-Functional Requirements Summary

Category	Specification
Performance	Dashboard loads < 2s (1,000 entries); ML scoring < 5s (500 entries); AI response < 8s. API endpoints < 2s under normal PoC load.
Security	bcrypt hashing (cost ≥ 10); PDO parameterised queries (SQL injection prevention); server-side PHP sessions; no sensitive data in client storage.
Maintainability	Strict Controller $\rightarrow$ Service $\rightarrow$ Model hierarchy; PSR-4 namespacing; documented API endpoints; no layer skipping.
Deployability	Single XAMPP installation; no containerisation required for PoC phase; Python and Java JDK as explicit prerequisites.

Table 3: Non-Functional Requirements Summary

3.3 User Roles and Permissions

Actor	Type	Core Permissions
Visitor	Unauthenticated	Browse public content, register, log in.
Learner	Human	Study materials, personal notes, read-only alerts, AI assistant.
Blue Team Operator	Human	Dashboard, log ingestion, alert management, ML scores, AI assistant.
Red Team Operator	Human	Port scanner, scan results, writeup publishing.
Purple Team Operator	Human (hybrid)	Combined Blue + Red capabilities, cross-correlation.
Administrator	Human	Full access: user management, system configuration, all modules.
Detection Engine	System actor	Autonomous ML scoring and alert generation (no user interaction).

Table 4: User Roles and System Actors

4. Use Case Modelling

4.1 Global Use Case Diagram

The following diagram captures all 33 use cases across seven functional packages, all eight actors, and the complete set of «include» and «extend» relationships. It is rendered on a dedicated portrait page to preserve readability (see [Figure 1](#)).

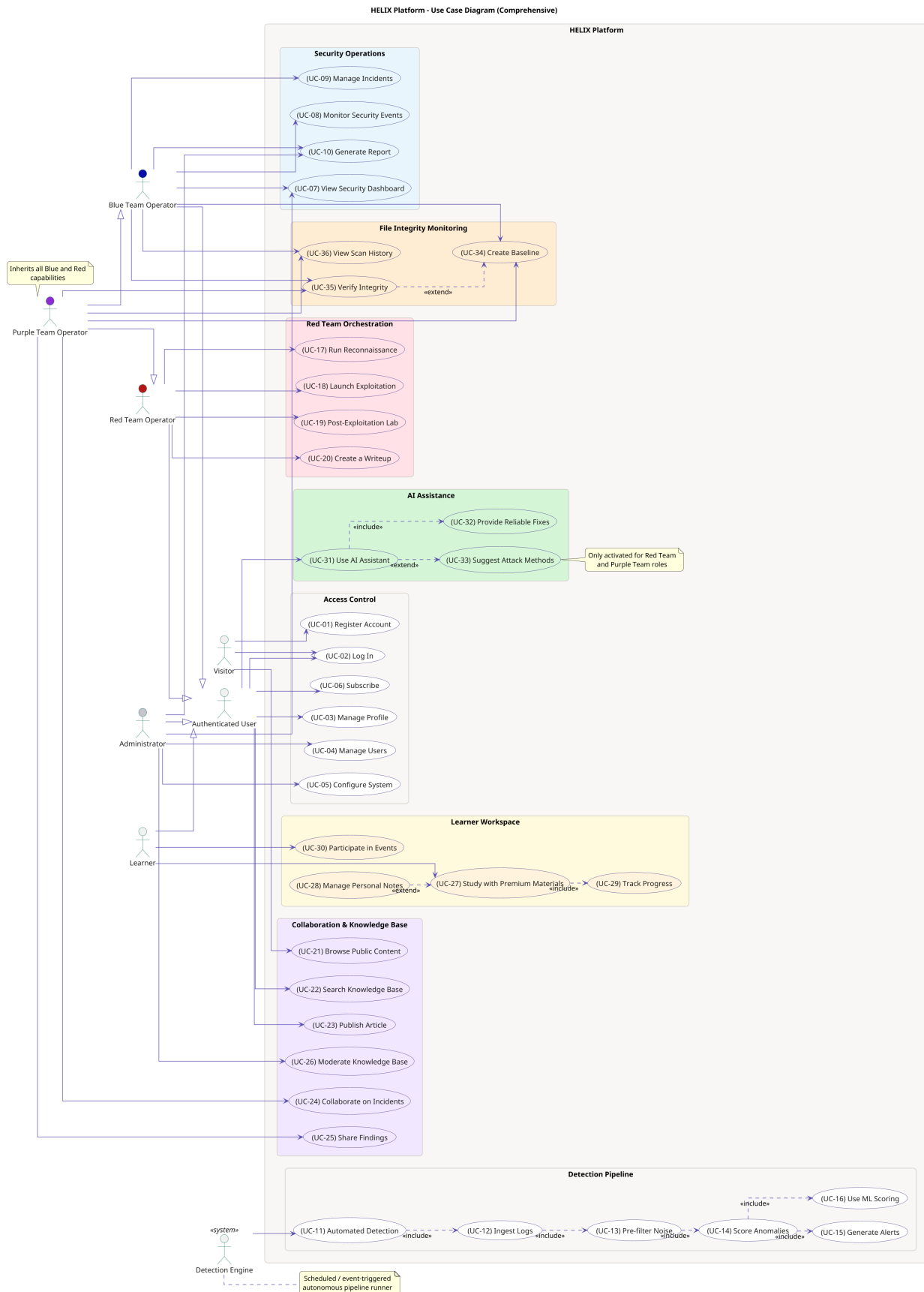


Figure 1: HELIX PoC — Global Use Case Diagram (all actors, all 33 UCs, all relationships)

4.2 Major Use Case Descriptions

UC-02: Log In

Actors: Visitor (initiates); Authenticated User (post-condition). **Preconditions:** Active account exists.

Main Flow: Submit credentials → Validate bcrypt hash → Create PHP session with role → Redirect to role-appropriate dashboard.

Alt Flow: Invalid credentials return HTTP 401 (no field disambiguation). **Postconditions:** Authenticated session established.

UC-11: Automated Detection

Actors: Detection Engine (system actor). **Preconditions:** Log entries have been ingested.

Main Flow: Trigger → *Ingest Logs* (UC-12) → *Pre-filter Noise* (UC-13) → *Score via ML* (UC-14/16) → *Generate Alerts* (UC-15) for HIGH/CRITICAL scores.

Alt Flow: ML service unavailable — entries queued for deferred scoring. **Postconditions:** Alerts generated and persisted.

UC-17: Run Reconnaissance

Actors: Red Team Operator, Purple Team Operator. **Preconditions:** Authenticated; target and port range specified.

Main Flow: Submit parameters → JavaBridge invokes Java scanner via `shell_exec()` → Parse output → Persist in `scan_results`.

Alt Flow: Unreachable target returns error; nothing persisted. **Postconditions:** Scan results stored and visible in history.

UC-31: Use AI Assistant

Actors: All authenticated roles. **Preconditions:** Authenticated; OpenAI API key configured.

Main Flow: Submit message (optionally with alert context) → Retrieve conversation history → Build prompt → Call OpenAI API → Persist and return response (*includes* UC-32: Reliable Fixes, UC-33: Suggest Attack Methods).

Alt Flow: API unavailable — degraded-mode message; session preserved. **Postconditions:** Guidance persisted; multi-turn conversation maintained.

5. Static Design

5.1 Class Diagram

The class diagram (Figure 2) covers all major architectural layers: Config, Middleware, Controllers, Services, Models, and Helpers. The strict dependency direction (Controller $\rightarrow$ Service $\rightarrow$ Model $\rightarrow$ Database) is enforced throughout.

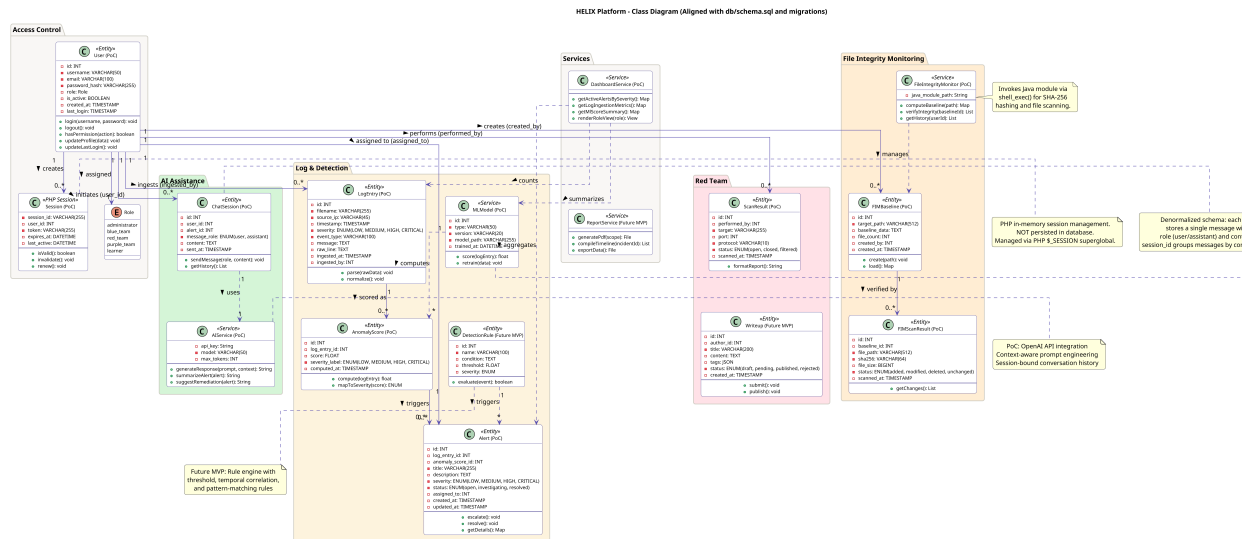


Figure 2: HELIX PoC — Class Diagram

5.2 Main Classes and Responsibilities

- **User** — Central entity. Attributes: `id`, `username`, `email`, `password_hash`, `role`, `status`. Operations: `findByUsername()`, `create()`, `updateRole()`, `toggleStatus()`. Extended by role-specific sub-entities.
- **LogEntry** — Parsed log record. Attributes: `id`, `timestamp`, `source_ip`, `severity`, `raw_message`, `parsed_fields`. Supports `bulkInsert()` for CSV batch ingestion.
- **Alert** — Generated from anomalous `LogEntries`. Attributes: `id`, `log_entry_id`, `severity_score`, `status` (Open / Investigating / Resolved), `created_at`.
- **AnomalyScore** — Persists ML detection results. Attributes: `id`, `log_entry_id`, `score`, `model_version`. Links to `LogEntry` by foreign key.
- **AIService** — Constructs multi-turn OpenAI prompts from `ChatSession` history, calls the external API, and persists responses in `ChatMessage`.
- **JavaBridge** — PHP-to-Java interoperability layer. Constructs `shell_exec()` commands for the compiled Java module and parses stdout.
- **ScanResult** — Stores port scanner output. Attributes: `id`, `target_ip`, `port`, `protocol`, `status`, `scan_date`, `operator_id`.

Key relationships: `User` 1 → * `Alert` (operator manages alerts); `LogEntry` 1 → 0..1 `AnomalyScore`; `AnomalyScore` 1 → 0..1 `Alert`; `User` 1 → * `ScanResult`; `ChatSession` 1 → * `ChatMessage`.

6. Dynamic Design

All five sequence diagrams share a common structure: Browser → `index.php` (front controller) → Middleware → Controller → Service → Model → MySQL, with external calls to Python, Java, or the OpenAI API where applicable.

6.1 Authentication Workflow

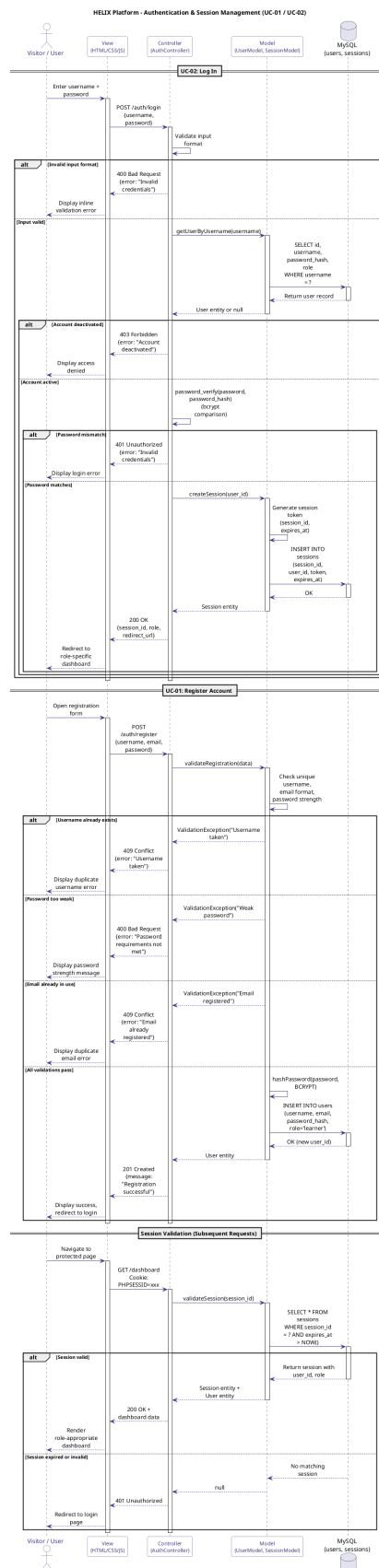


Figure 3: Sequence Diagram: Authentication and Role-Based Session Establishment

The user submits credentials to `POST /auth/login`. `AuthService` queries `User::findByUsername()` via PDO, verifies the bcrypt hash with `password_verify()`, and populates the PHP session with the user ID and role. On failure, HTTP 401 is returned with no field-specific information.

Figure 4: Sequence Diagram: Security Dashboard and Event Monitoring

The dashboard sequence captures role-aware metrics aggregation, alert summaries, and filtered event queries. It is kept on a portrait page because the rendered asset is tall rather than wide.

6.3 Log Ingestion and ML Detection Pipeline

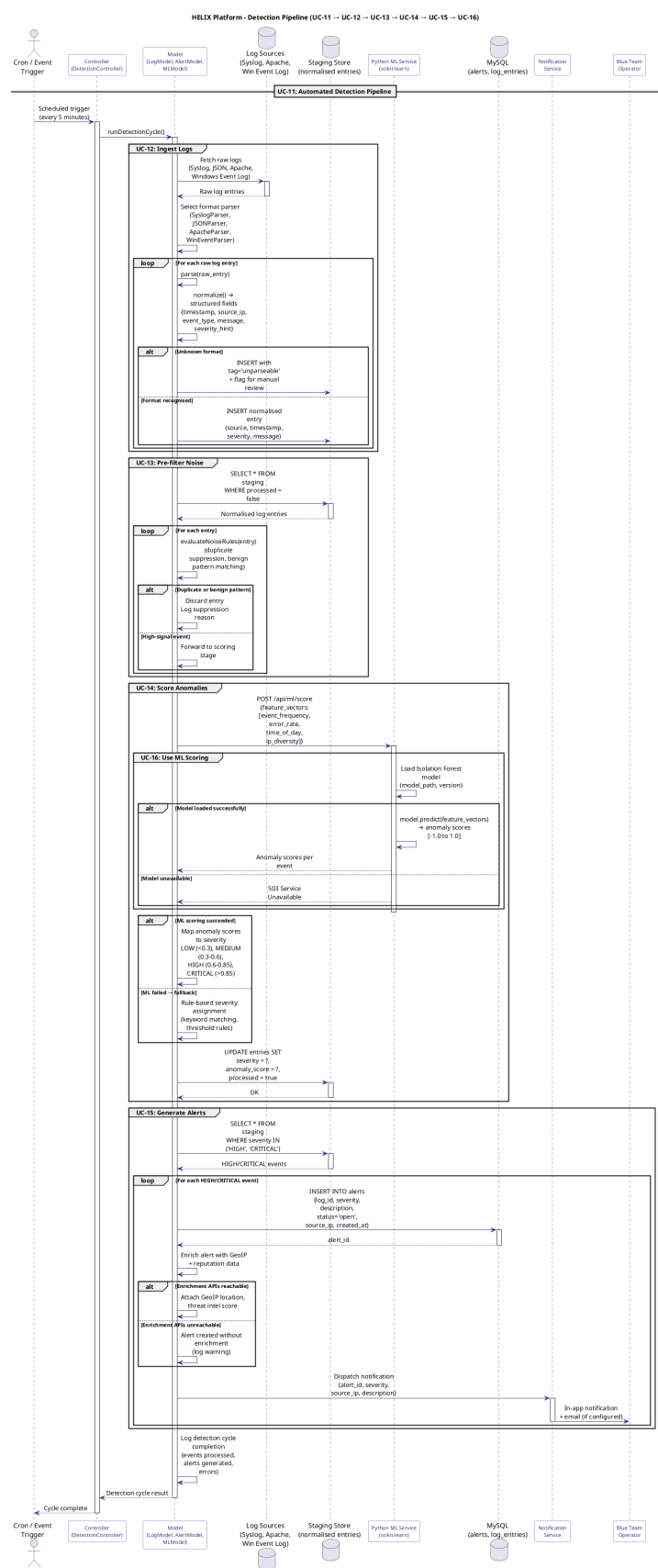
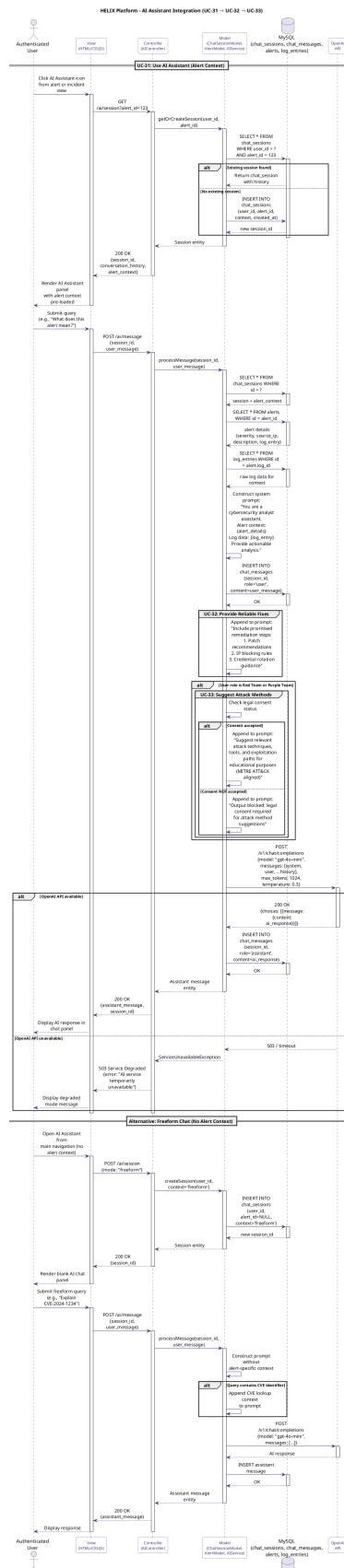


Figure 5: Sequence Diagram: Log Ingestion and Automated Detection Pipeline

The operator uploads a CSV/TXT file. After role validation, the ingestion service parses rows, stores them, and runs the detection cycle. The backend then invokes the Python detection script via `shell_exec()`, passing feature vectors as JSON. Python applies Isolation Forest, returns scores to stdout, and the PHP layer persists them via `AnomalyScore::insertBatch()`. Entries exceeding the HIGH/CRITICAL threshold trigger `Alert::create()`.

6.4 AI Assistant Workflow



The user sends a message with optional alert context. **AIService** retrieves conversation history from **ChatMessage**, constructs a multi-turn prompt with a cybersecurity-specialist system role, and dispatches an HTTPS call to the OpenAI API. The response is simultaneously returned to the frontend and persisted.

6.5 Red Team Reconnaissance Workflow

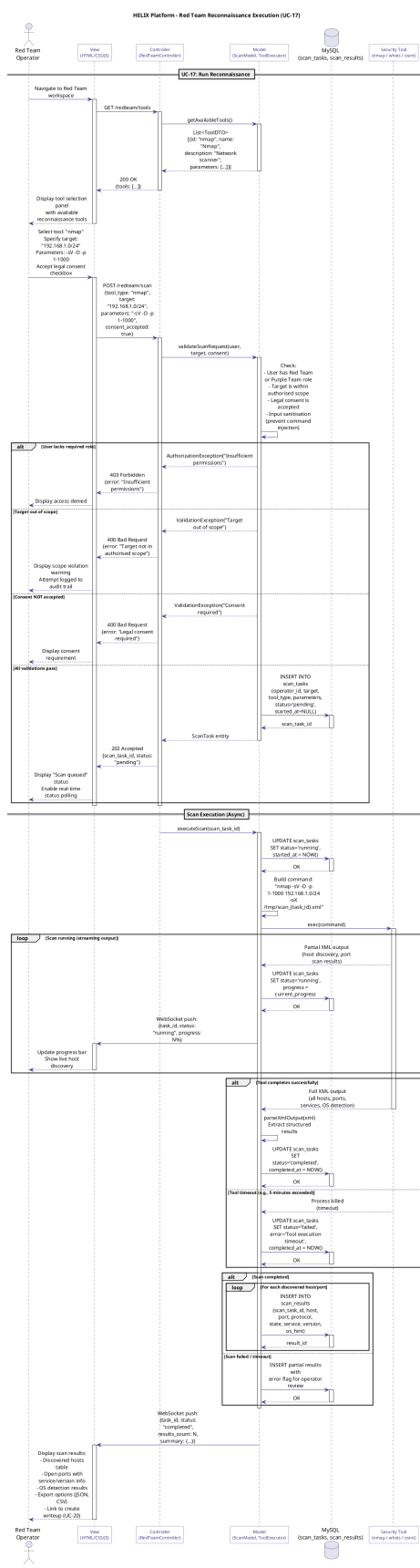


Figure 7: Sequence Diagram: Red Team Port Scanner — Java Module Execution

An authorised Red or Purple Team operator submits a target IP and port range. `JavaBridge::execute()` constructs a `java -cp ... PortScanner` command via `shell_exec()`. The Java module scans and writes structured results to stdout; the bridge persists them in `ScanResult`.

7. Business Process Modelling

The three activity diagrams below model the principal operational workflows. Each rendered PNG is placed on its own page because the generated diagrams are already optimized for readability rather than compactness.

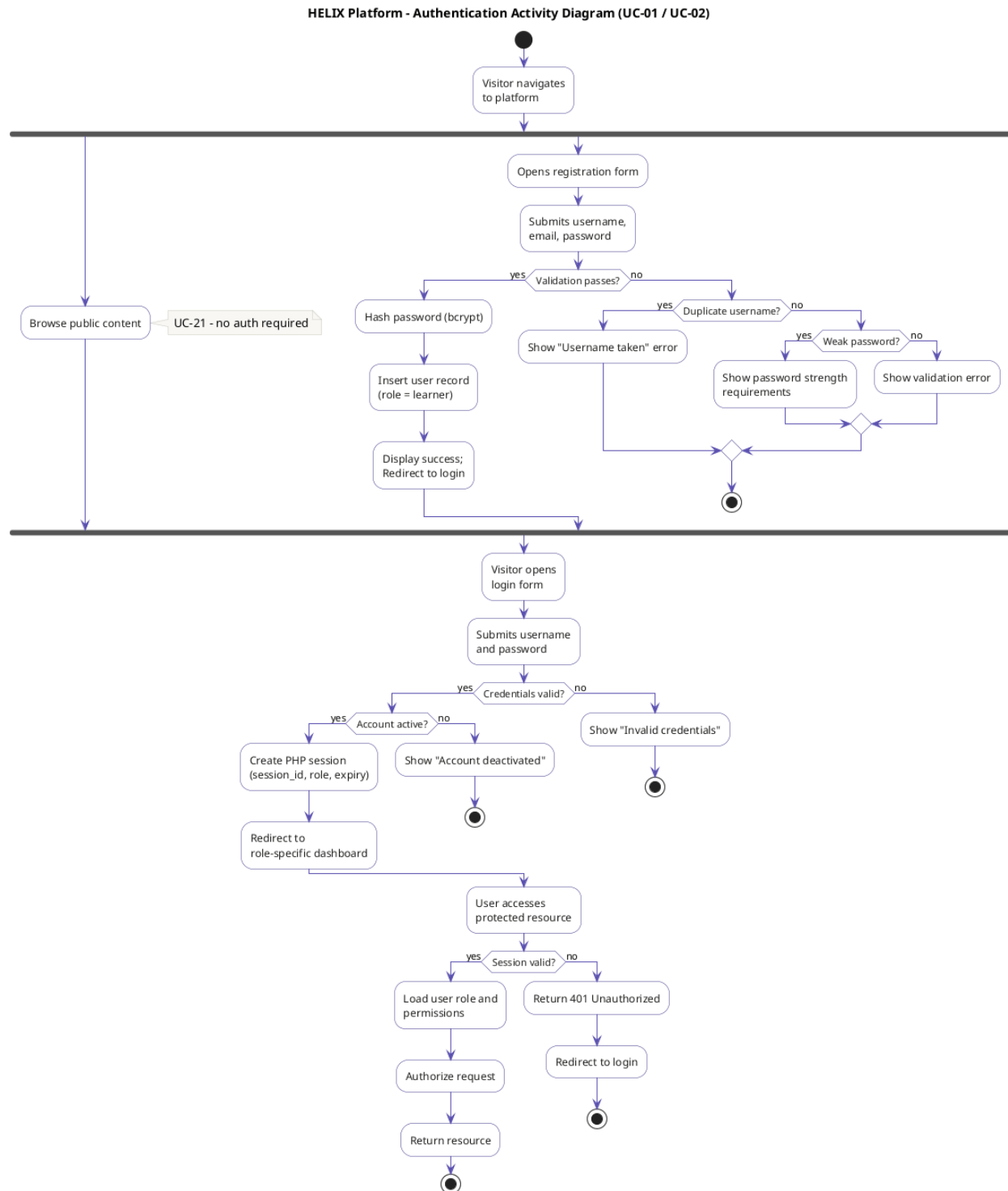


Figure 8: Activity: Authentication Process

Authentication (Figure 8): Three sequential guard conditions — input validation, user lookup, bcrypt verification. Any failure produces a generic error (no field disambiguation). Success initialises the PHP session.

HELIX Platform - Alert Lifecycle Activity Diagram

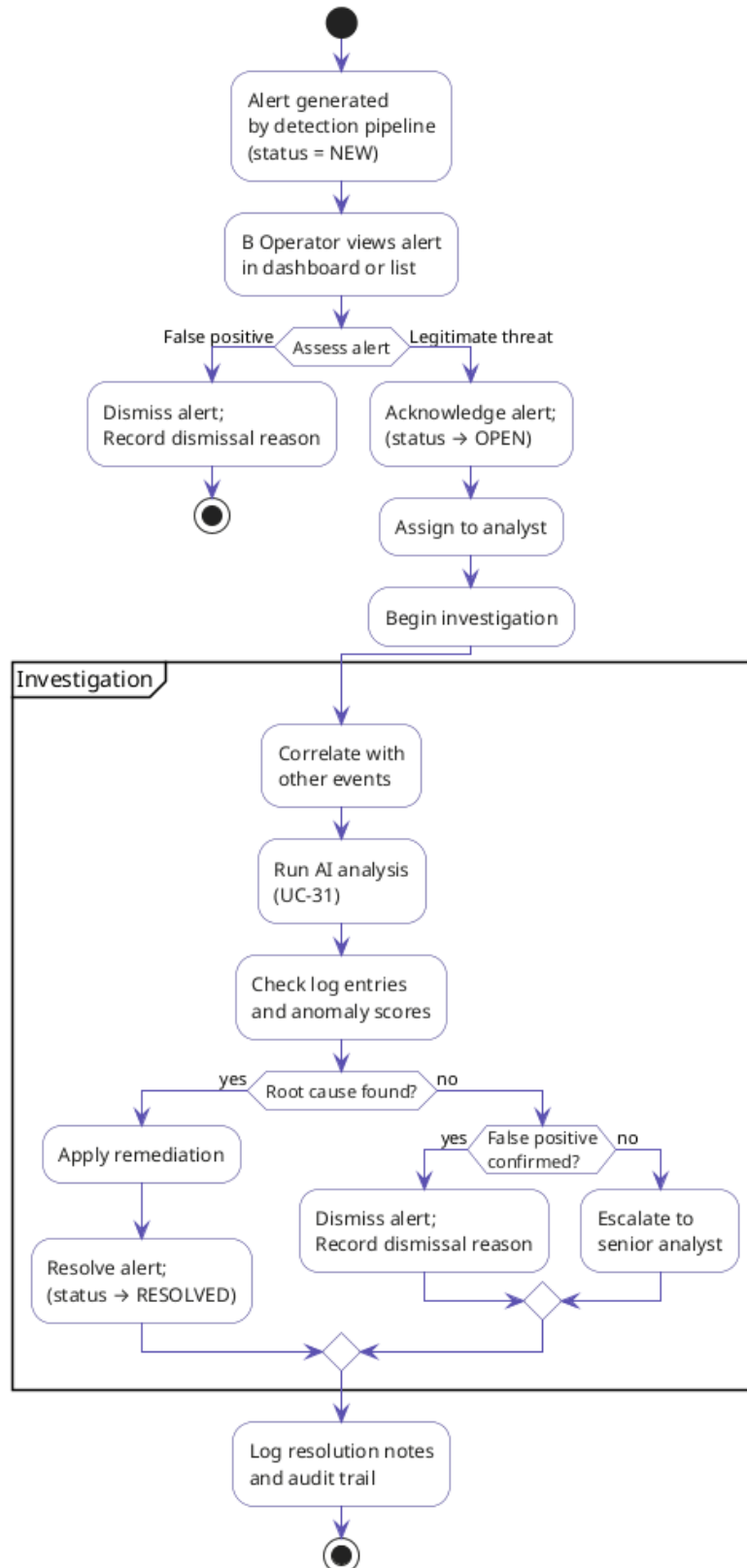


Figure 9: Activity: Alert Lifecycle and Triage Process

Alert Lifecycle (Figure 9): This workflow covers acknowledgement, investigation, escalation, and resolution. It complements the detection pipeline by showing how generated alerts move

through analyst review.

HELIX Platform - Detection Pipeline Activity Diagram (UC-11 → UC-15)

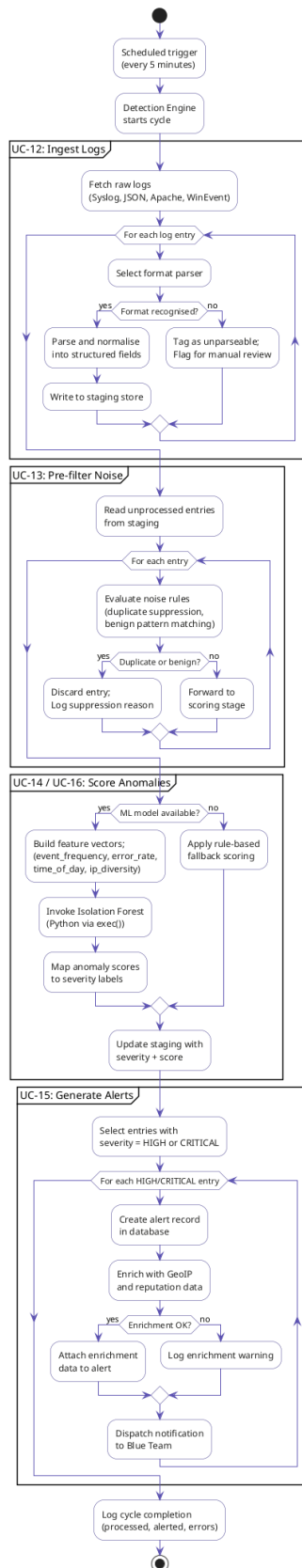


Figure 10: Activity Diagram: Log Ingestion and Automated Detection Pipeline

Detection Pipeline (Figure 10): After ingestion, the flow forks — database persistence

and ML scoring proceed concurrently. Entries exceeding the anomaly threshold follow the alert generation branch; compliant entries are marked scored. A deferred-scoring path handles ML service unavailability.

8. Software Architecture

8.1 Architectural Overview

The HELIX PoC follows a **Modular Monolith** architecture with clear service boundaries, designed for simplicity in the PoC phase whilst enabling seamless extraction of the ML, AI, and Java services into independent microservices for the MVP phase.

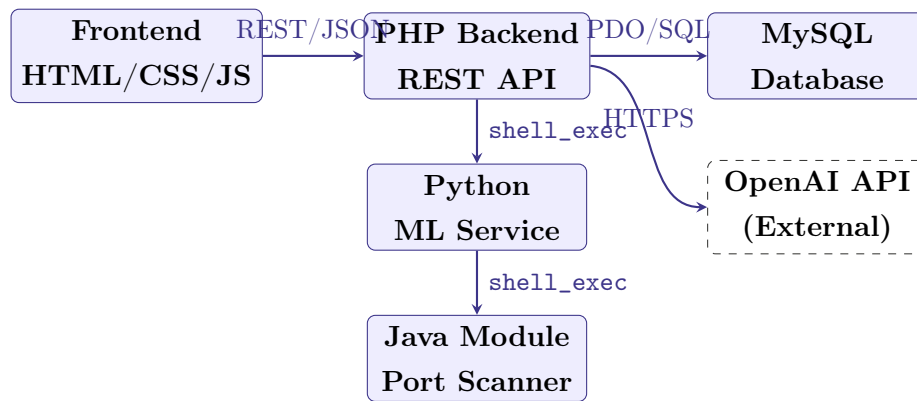


Figure 11: Component Interaction Overview — HELIX PoC

8.2 Major Components

1. **Presentation Layer:** HTML/CSS/JS frontend providing role-based dashboards, alert views, log upload forms, and AI chat interface. Communicates exclusively with the PHP backend via the JSON REST API.
2. **Application Layer (PHP):** Layered REST API. Handles HTTP requests, session management, RBAC enforcement via middleware, and orchestration of calls to external services. Provides the `JavaBridge` abstraction for subprocess-based integrations.
3. **Data Layer (MySQL):** Seven relational tables: `users`, `log_entries`, `alerts`, `anomaly_scores`, `chat_sessions`, `chat_messages`, `scan_results`. All queries use PDO parameterised statements.
4. **ML Service (Python):** Standalone script (`ml_service/detect.py`) invoked synchronously via `shell_exec()`. Applies the Isolation Forest model to log feature vectors and returns anomaly scores as JSON to stdout.

5. **AI Service (External):** OpenAI Chat Completions API. Called from `AIService` with a system prompt, conversation history, and user message. Responses are persisted in `chat_messages` for multi-turn continuity.
6. **Red Team Module (Java):** Compiled Java class files invoked through `JavaBridge`. Performs port scanning and hash analysis with structured stdout output captured and persisted by the PHP layer.

Interaction principle: The PHP backend is the sole orchestrator. Neither the frontend nor subprocess modules access MySQL directly. All inter-service communication is mediated by the service layer, ensuring consistent audit logging, input validation, and data formatting before persistence.

9. Deployment Architecture

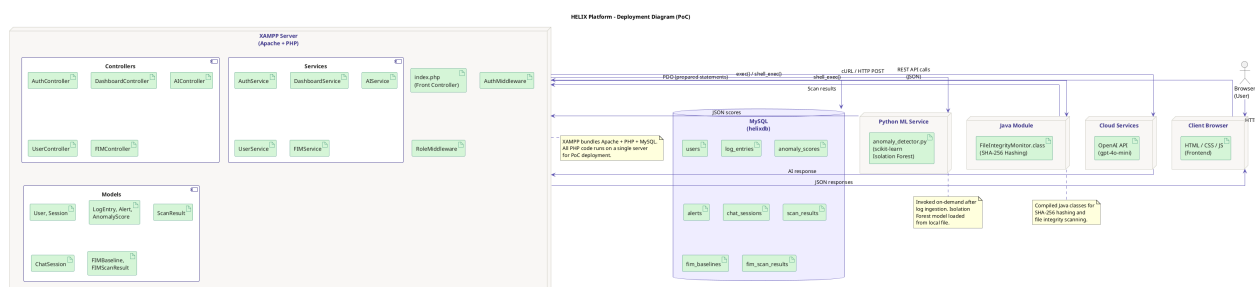


Figure 12: Deployment Diagram: PoC Execution Environment and Node Topology

9.1 Execution Environment and Node Responsibilities

- **Client Node:** Any modern web browser (Chrome, Firefox, Edge) accessing `http://helix.local` or `http://localhost/HELIX`. Renders the frontend interface and sends HTTP/JSON requests to the backend.
- **Application Server (XAMPP / localhost):** Hosts Apache 2.x, PHP 8.x, the HELIX application code, the Python runtime, and the Java JDK. Manages all user sessions, API routing, and subprocess invocations. MySQL 8.x co-located on the same node in the PoC configuration.
- **Database Server (MySQL):** Co-located with the application server. Stores all persistent data; accessible only from `localhost` (no external network exposure).
- **ML Execution Environment:** Python 3.x with scikit-learn and pandas, installed on the

same server node. Invoked synchronously on demand — no persistent process or queue required.

- **External Cloud Services:** OpenAI API (`api.openai.com`) accessed via HTTPS from the PHP backend. The only external runtime dependency; requires an active internet connection and a valid API key stored in `.env`.

Source	Target	Protocol	Notes
Browser	Apache/PHP	HTTP/1.1	REST JSON; session via cookie
PHP	MySQL	TCP/3306 (PDO)	Parameterised queries; localhost only
PHP	Python ML	<code>shell_exec</code>	JSON via <code>stdin/stdout</code> ; synchronous
PHP	Java Module	<code>shell_exec</code>	Args + <code>stdout</code> ; synchronous
PHP	OpenAI API	HTTPS/443	API key in Authorization header

Table 5: Inter-Node Communication Protocols

10. Conclusion

10.1 Summary of the PoC

The HELIX Proof of Concept delivers a fully functional, multi-module cybersecurity operations platform across seven integrated capabilities — authentication and RBAC, security dashboard, log ingestion, alert visualisation, ML anomaly detection, AI-assisted analysis, and Red Team orchestration. The platform is implemented on a deliberately accessible stack (PHP, MySQL, Python, Java) deployable on a single XAMPP server, demonstrating that architectural coherence and academic rigour need not depend on complex infrastructure.

10.2 Achieved Objectives

- Implemented robust RBAC across six human roles and one system actor, with per-layer enforcement at the middleware and route levels.

- Established a functional log ingestion and ML detection pipeline from CSV upload to anomaly score persistence and threshold-based alert generation.
- Integrated the Isolation Forest algorithm for unsupervised anomaly detection without labelled training data.
- Demonstrated practical AI integration (OpenAI API) for alert explanation and multi-turn remediation guidance with conversation persistence.
- Provided a controlled, audited Red Team environment for port scanning, restricted to authorised roles.
- Produced a complete documentation suite: SRS, 33 use case descriptions, full UML artefact set, and this SDD.

10.3 Architectural Strengths

The modular monolith architecture ensures straightforward XAMPP deployment while maintaining strict separation of concerns across all layers. The decoupling of ML and AI services via well-defined subprocess and HTTP interfaces means these components can be migrated to independent microservices in the MVP phase without altering the core application logic. Security is enforced by construction — bcrypt hashing, parameterised queries, and server-side session state are architectural invariants, not optional policies. The resulting platform serves as a solid, extensible foundation for the planned MVP phase, which will introduce containerisation, real-time event streaming, and advanced exploit simulation.